I. Problema del cambio de monedas

I.D Análisis experimental:

Sistema de monedas	Optimal	Suboptimal	Greedy_failed	Impossible	T. Greedy	T. Óptimo
[200, 100, 50, 20, 10, 5, 2, 1]	31 (6.2%)	469 (93.8%)	0 (0.0%)	0 (0.0%)	0.00064	0.27107
[10, 5, 2, 1]	8 (1.6%)	492 (98.4%)	0 (0.0%)	0 (0.0%)	0.00073	0.06917
[25, 10, 4, 1]	10 (2.0%)	490 (98.0%)	0 (0.0%)	0 (0.0%)	0.00047	0.06774
[4, 3, 1]	5 (1.0%)	495 (99.0%)	0 (0.0%)	0 (0.0%)	0.00095	0.05037

Como se puede ver en los datos de las dos últimas columnas, a pesar de que el algoritmo greedy es menos eficiente (no produce necesariamente la solución óptima al problema) es varios órdenes mayor en coste de tiempo (aproximadamente 10^3 órdenes más rápido).

Cuestiones:

1. ***¿Cuál es el criterio codicioso utilizado?***

El criterio codicioso se basa en ir seleccionando en cada paso la moneda de mayor valor que no supere la cantidad restante por completar. Esto implica que la decisión se toma de manera local y es irreversible, es decir, se toma como cierta hasta el final de la ejecución del algoritmo.

2. ***¿Funciona correctamente para todos los sistemas monetarios?***

Funciona correctamente únicamente para los sistemas monetarios canónicos. Los sistemas monetarios canónicos son aquellos que permiten alcanzar la menor cantidad de monedas posibles para un cambio (sistema euro o dólar).

3. ***¿Cuál es la complejidad temporal de la solución codiciosa?***

La complejidad temporal total se encuentra en O (m* log m +k).

4. ***¿Cuál es la complejidad temporal y espacial de la programación dinámica?***

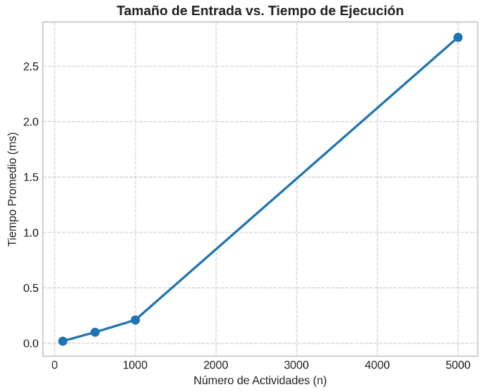
Teniendo una matriz de m tipos de monedas y una cantidad A: la complejidad temporal será m*A y la complejidad espacial será también m*A (ya que se necesita m para reconstruir).

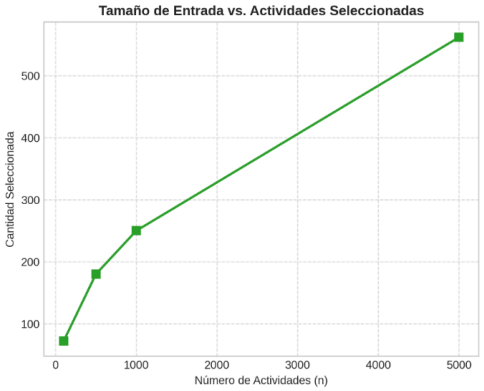
5. ***¿Qué ventajas presenta la solución codiciosa, aunque no siempre garantice el óptimo?***

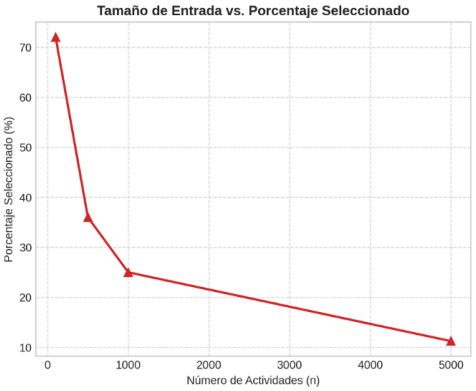
Las principales ventajas son la eficiencia temporal y espacial, como se deja claro en el anterior apartado. La solución codiciosa depende linealmente de m (mientras que la dinámica depende de m*A) por lo que temporalmente es mucho más rápida y por lo tanto únicamente tiene una dimensión en la matriz que necesita para guardar, lo que lo hace mejor en términos de almacenamiento y procesamiento en memorias.

II. Selección de actividades

II.C Análisis experimental:







Cuestiones:

1. ***¿Cuál es el criterio codicioso utilizado?***

El criterio codicioso se basa en ir seleccionando en cada paso la actividad compatible con menor coste de tiempo de finalización. En caso de empate se ordena según el tiempo de inicio y posteriormente en orden lexicográfico, de menor a mayor. Esto implica que la decisión se toma de manera local y es irreversible, es decir, se toma como cierta hasta el final de la ejecución del algoritmo.

2. ***¿Por qué seleccionar la actividad que termina antes deja más posibilidades?***

Al elegir la actividad que termina más temprano se optimizan los recursos computacionales, ya que se selecciona nada más terminar y no hay que operar más con ese dato, minimizando el coste temporal de cada operación. Esto implica que se deja más tiempo para la ordenación de las actividades posteriores.

3. ***¿Qué podría suceder si se eligiese primero la actividad que comienza antes?***

Esto produciría una ordenación subóptima del problema, ya que no se está teniendo en cuenta la duración de la actividad en sí. Por ejemplo, si la actividad que primero se inicia durase todo el conjunto de tiempo disponible, ello implicaría la selección de esta aunque hubiese "n" actividades de duración tiempo_total/n, es decir, una secuencia de actividades que tengan repartido todo el tiempo equitativamente (lo cual siempre será más optimo que la primera opción).

4. ***¿Qué podría suceder si se eligiese primero la actividad de menor duración?***

Esta idea también daría una solución subóptima, ya que no tiene en cuenta el solapamiento entre actividades. Un ejemplo sencillo que demuestra que no sería un criterio óptimo:

Teniendo el conjunto de tareas T = [(0,4) , (3,6) , (5, 9)] este criterio elegiría la tarea (3,6) ya que sería la de menor duración, sin embargo se puede ver a simple vista que no sería la solución óptima ya que [(0,3) , (5,9)] cumple todo el conjunto de manera eficiente.

5. ***¿Cuál es la complejidad temporal y espacial?***

   La complejidad temporal de la selección es de O (n * log n), que viene determinada por la ordenación inicial de la lista proporcionada de actividades. La complejidad espacial es de O (n), ya que incluye la lista inicial, la lista ordenada y la lista resultante.

III. Planificación de tareas con fechas límite

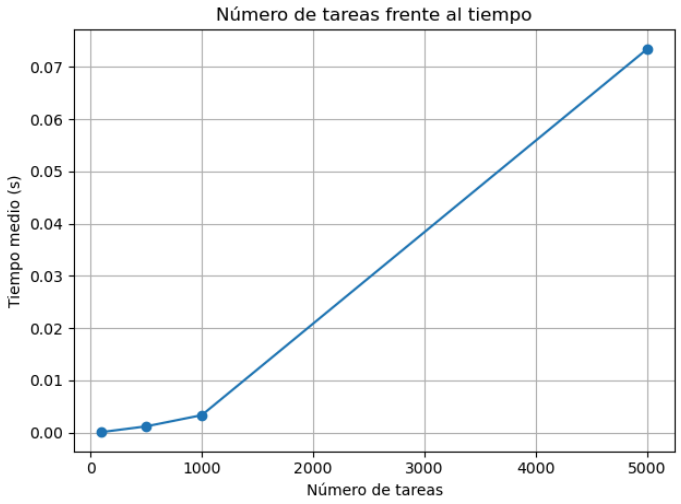
III.C Análisis experimental:

Al hacer las gráficas con los datos que me pedía, me han salido estos datos:

Tareas	Tiempo (s)	Beneficio	Seleccionadas	Porcentaje
100	0.000110	9578	10	10.00%
500	0.001206	47584	50	10.00%
1000	0.003352	94923	100	10.00%
5000	0.073445	473174	500	10.00%

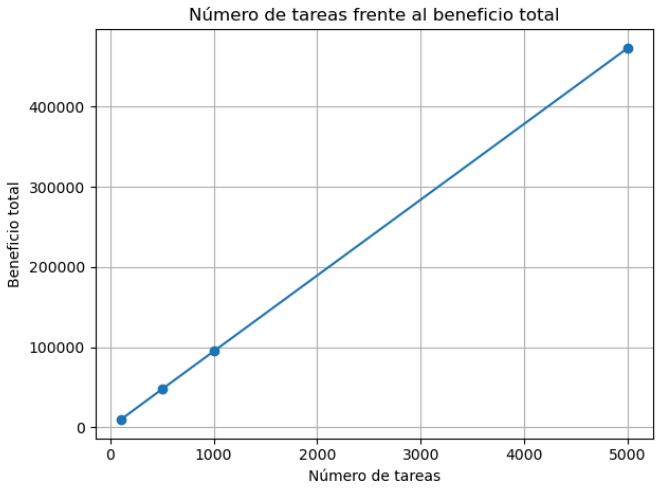
Y las gráficas son estas:

- Gráfica del número de tareas frente al tiempo.



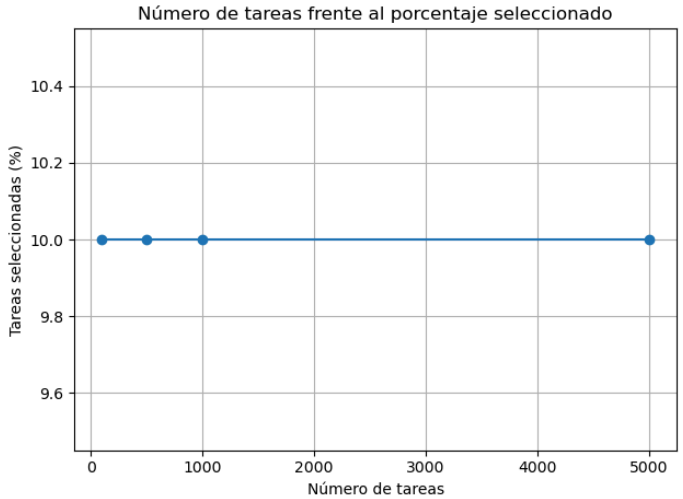
Muestra cómo crece el tiempo de ejecución a medida que aumenta el volumen de tareas. Aunque el algoritmo ordena los elementos inicialmente en $O(n\log n)$, el bucle de asignación anidado busca slots libres hacia atrás. Al estar el calendario altamente saturado, las tareas evaluadas al final de la lista deben recorrer muchos slots ya ocupados antes de ser descartadas, lo que acentúa una tendencia que empíricamente se aproxima a un comportamiento cuadrático $O(n * \text{max_deadline}) \approx O(n^2)$ en el peor de los casos. El salto a 5000 tareas refleja claramente este incremento (0.073 segundos).

- Gráfica del número de tareas frente al beneficio total.



Presenta un crecimiento estrictamente lineal. Debido a que el porcentaje de éxito es constante (10.00%) y los beneficios se distribuyen uniformemente entre 1 y 1000 (con una media teórica de 500.5 por tarea), el beneficio acumulado es directamente proporcional al tamaño de la muestra. Al seleccionar siempre el mejor 10% de las tareas disponibles, a mayor n, mayor es el número de tareas de alto beneficio que logran entrar en el esquema.

- Gráfica del número de tareas frente al porcentaje seleccionado.



La gráfica muestra un comportamiento completamente plano y constante en el 10.00% para todos los volúmenes de tareas evaluados. Esto refleja visualmente que el algoritmo alcanza siempre el límite

máximo de ocupación permitido por el generador de pruebas. Al haber exactamente diez veces más tareas que huecos temporales disponibles en el calendario, el sistema se satura por completo en cada experimento, logrando ubicar únicamente la décima parte de la carga total de trabajo.

Cuestiones:

1. ***¿Cuál es el criterio codicioso utilizado?***

El criterio codicioso

2. ***¿Por qué se procesan primero las tareas con mayor beneficio?***

Porque el objetivo del problema es maximizar el beneficio total acumulado, y como hay recursos, y en este caso tiempo limitado, hay que procesar primero las tareas más valiosas para no perder los mayores ingresos.

3. ***¿Cómo afecta la distribución de deadlines?***

Determina la capacidad total del calendario y el nivel de colisión. En este experimento, al estar acotada a la décima parte de las tareas (n//10), genera una saturación muy grande que obliga a descartar el 90% de ellas. Si los *deadlines* fueran mayores, el porcentaje de éxito subiría al haber más huecos disponibles, y por tanto habría más beneficio.

4. ***¿Qué podría suceder si se ordenasen únicamente por deadline?***

Se priorizarían las tareas que vencen antes. Esto permitiría salvar más cantidad de tareas, pero no se conseguiría maximizar el beneficio total, pudiendo llenar el calendario con tareas de valor mínimo (ej. 1) y descartando las más lucrativas (ej. 1000).

5. ***¿Cuál es la complejidad temporal y espacial?***

La complejidad temporal es de $O(n^2)$ en el peor de los casos, mientras que la ordenación inicial cuesta $O(n \log n)$ como ya expliqué antes en la gráfica de Número de Tareas frente al Tiempo.

La complejidad espacial es de $O(n)$, ya que el espacio crece de forma estrictamente lineal para almacenar la lista de tareas recibidas, su copia ordenada y el vector del calendario, como ya expliqué en la gráfica de Número de Tareas frente al Beneficio Total.

IV. Cuestiones generales

1. ***¿Qué rasgos permiten afirmar que los tres algoritmos estudiados pertenecen al paradigma codicioso?***

Los tres algoritmos (cambio de monedas, selección de actividades y planificación de tareas) comparten los siguientes rasgos fundamentales del algoritmo greedy :

Elecciones locales: En cada paso del algoritmo se toma una decisión basándose en la mejor opción en ese momento (por ejemplo, la moneda de mayor valor, la actividad que termina antes, o la tarea con mayor beneficio).

Irrevocabilidad: Una vez se toma la decisión sobre un elemento, se considera como cierta hasta el final del proceso.

Construcción iterativa: La solución se construye a base de repeticiones sobre una misma idea.

2. ***¿Qué diferencias existen entre los tres algoritmos en complejidad temporal y espacial?***

Algoritmo	**Complejidad Temporal**	**Complejidad Espacial**
Cambio de monedas	O (m*log m + k)	O (k)
Selección de actividades	O (n * log n)	O (n)
Planificación de tareas	O (n * log n + D)	O (n + D)